

Operating Systems Project Report

Project Title: PC Bootstrap and Minimal MBR Implementation

1. Introduction

In this project, I implemented the boot process of a PC from scratch. The primary goal was to understand how control is handed over from the hardware (Firmware) to a software program through the BIOS and a custom-made Bootloader.

2. Implementation Details

- **Source Code:** I created a file named `boot.asm` using Assembly language with Intel syntax.
- **16-bit Real Mode:** The directive `bits 16` was used because modern X86 processors start up pretending to be a 16-bit 8086 processor for backward compatibility.
- **Compilation:** I used the **NASM** assembler to convert the code into a plain binary file named `boot.bin`.
- **Emulation:** The binary was tested using the **QEMU** emulator (via the v86 web interface), which simulates X86 hardware.

3. Core Components of the Code

The following elements were essential for a successful boot:

- **Memory Segmentation:** The data segment register (`ds`) was pointed to `0x07C0` because the BIOS always loads the MBR into memory address `0x07C00`.
- **BIOS Interrupts:** I utilized the BIOS system call `int 0x10` with function `0x0E` in the `AH` register to print the "Hello" message character by character to the screen.
- **Magic Signature:** The last two bytes of the 512-byte sector were set to `0xAA55`. This "magic signature" is required for the BIOS to recognize the disk as bootable.

4. Results

Upon running the simulation, the BIOS successfully located my MBR, verified the magic signature, and executed the code. A terminal window appeared displaying the **"Hello"** message, confirming that the custom bootloader is functional.

5. Conclusion

This phase demonstrates that there is "no magic" to operating systems. It is a well-defined sequence of instructions starting from the firmware and leading to full control over the hardware.
